

My Project

Generated by Doxygen 1.8.11

Contents

1	Assembleur	1
1.1	Introduction	1
1.2	Installation	1
1.2.1	Step 1: Opening the box	1
2	Class Index	3
2.1	Class List	3
3	File Index	5
3.1	File List	5
4	Class Documentation	7
4.1	instructionName Struct Reference	7
4.1.1	Member Data Documentation	7
4.1.1.1	format	7
4.1.1.2	name	7
4.1.1.3	nbops	7
4.1.1.4	opcode	7
4.1.1.5	type	7
4.2	label Struct Reference	7
4.2.1	Member Data Documentation	8
4.2.1.1	addr	8
4.2.1.2	label	8
4.3	ref Struct Reference	8
4.3.1	Member Data Documentation	8
4.3.1.1	addrInCode	8
4.3.1.2	label	8

5	File Documentation	9
5.1	asm.c File Reference	9
5.1.1	Function Documentation	10
5.1.1.1	addCode(int v)	10
5.1.1.2	addInstructionName(char *instname, int opcode, int type, char *format, int nbops)	10
5.1.1.3	addLabel(char *labelname, int addr)	10
5.1.1.4	addReference(char *labelname, int addrInCode)	11
5.1.1.5	decodeInstruction(char *line)	11
5.1.1.6	dumpBinaryCode()	12
5.1.1.7	findLabel(char *labelname)	12
5.1.1.8	generateBinary(FILE *fout)	12
5.1.1.9	main(int argc, char **argv)	13
5.1.1.10	parseAsm(FILE *fin)	13
5.1.1.11	printLabels()	14
5.1.1.12	resolveReferences()	14
5.2	asm.h File Reference	15
5.2.1	Macro Definition Documentation	16
5.2.1.1	MAX_CODESEGMENT_SIZE	16
5.2.1.2	MAX_IDENTS_SIZE	16
5.2.1.3	MAX_LABELS_SIZE	16
5.2.2	Function Documentation	16
5.2.2.1	addCode(int v)	16
5.2.2.2	addInstructionName(char *instname, int opcode, int type, char *format, int nbops)	16
5.2.2.3	addLabel(char *labelname, int addr)	17
5.2.2.4	addReference(char *labelname, int addrInCode)	17
5.2.2.5	decodeInstruction(char *line)	18
5.2.2.6	dumpBinaryCode()	18
5.2.2.7	findLabel(char *labelname)	19
5.2.2.8	generateBinary(FILE *fout)	19
5.2.2.9	parseAsm(FILE *fin)	19
5.2.2.10	printLabels()	20
5.2.2.11	resolveReferences()	20
5.2.3	Variable Documentation	21
5.2.3.1	codeSegment	21
5.2.3.2	currentInst	21
5.2.3.3	currentInstructionName	21
5.2.3.4	currentLabel	21
5.2.3.5	currentRef	21
5.2.3.6	tabInstructionNames	21
5.2.3.7	tabLabels	21
5.2.3.8	tabReferences	21

Chapter 1

Assembleur

1.1 Introduction

Appel du programme: `:asm fichierIn.asm fichierOut.bin`

Ce programme permet d'assembler un fichier assembleur en bytecode

1.2 Installation

1.2.1 Step 1: Opening the box

etc...

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

instructionName	7
label	7
ref	8

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

asm.c		9
asm.h		15

Chapter 4

Class Documentation

4.1 instructionName Struct Reference

```
#include <asm.h>
```

Public Attributes

- char * [name](#)
- int [opcode](#)
- int [type](#)
- char * [format](#)
- int [nbops](#)

4.1.1 Member Data Documentation

4.1.1.1 char* instructionName::format

4.1.1.2 char* instructionName::name

4.1.1.3 int instructionName::nbops

4.1.1.4 int instructionName::opcode

4.1.1.5 int instructionName::type

The documentation for this struct was generated from the following file:

- [asm.h](#)

4.2 label Struct Reference

```
#include <asm.h>
```

Public Attributes

- char * [label](#)
- int [addr](#)

4.2.1 Member Data Documentation

4.2.1.1 int `label::addr`

4.2.1.2 char* `label::label`

The documentation for this struct was generated from the following file:

- [asm.h](#)

4.3 ref Struct Reference

```
#include <asm.h>
```

Public Attributes

- char * [label](#)
- int [addrInCode](#)

4.3.1 Member Data Documentation

4.3.1.1 int `ref::addrInCode`

4.3.1.2 char* `ref::label`

The documentation for this struct was generated from the following file:

- [asm.h](#)

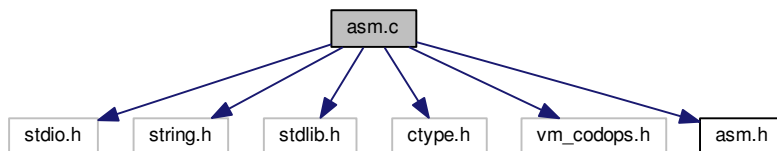
Chapter 5

File Documentation

5.1 asm.c File Reference

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>
#include "vm_codops.h"
#include "asm.h"
```

Include dependency graph for asm.c:



Functions

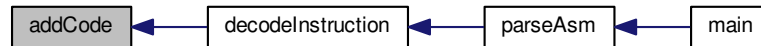
- void `addCode` (int v)
Complète le tableau codeSegment.
- void `addInstructionName` (char *instname, int opcod, int type, char *format, int nbops)
- int `findLabel` (char *labelname)
- void `addLabel` (char *labelname, int addr)
- void `addReference` (char *labelname, int addrInCode)
- void `resolveReferences` ()
- void `decodeInstruction` (char *line)
- int `parseAsm` (FILE *fin)
- void `printLabels` ()
- void `dumpBinaryCode` ()
- void `generateBinary` (FILE *fout)
- int `main` (int argc, char **argv)

5.1.1 Function Documentation

5.1.1.1 void addCode (int v)

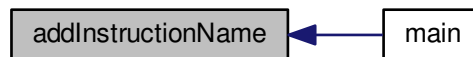
Complète le tableau codeSegment.

Here is the caller graph for this function:



5.1.1.2 void addInstructionName (char * instname, int opcode, int type, char * format, int nbops)

Here is the caller graph for this function:

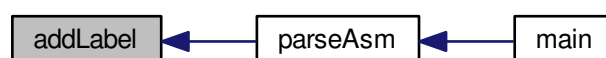


5.1.1.3 void addLabel (char * labelname, int addr)

Here is the call graph for this function:

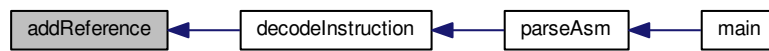


Here is the caller graph for this function:



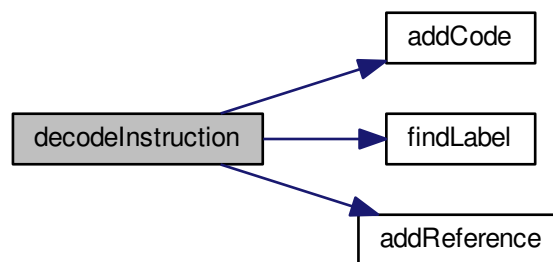
5.1.1.4 void addReference (char * *labelname*, int *addrInCode*)

Here is the caller graph for this function:

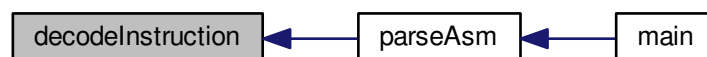


5.1.1.5 void decodeInstruction (char * *line*)

Here is the call graph for this function:

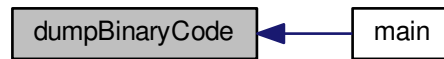


Here is the caller graph for this function:



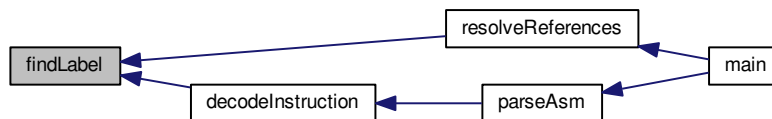
5.1.1.6 void dumpBinaryCode ()

Here is the caller graph for this function:



5.1.1.7 int findLabel (char * *labelname*)

Here is the caller graph for this function:



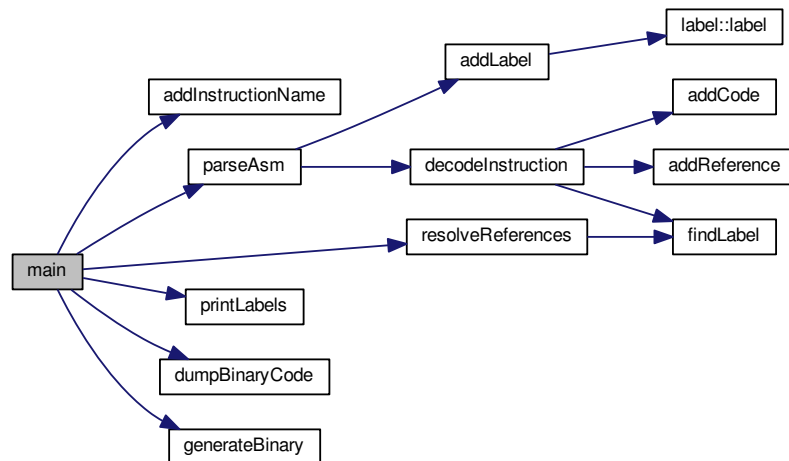
5.1.1.8 void generateBinary (FILE * *fout*)

Here is the caller graph for this function:



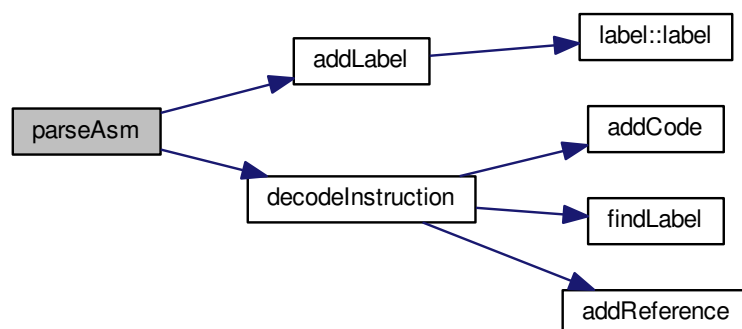
5.1.1.9 int main (int argc, char ** argv)

Here is the call graph for this function:



5.1.1.10 int parseAsm (FILE * fin)

Here is the call graph for this function:



Here is the caller graph for this function:



5.1.1.11 void printLabels ()

Here is the caller graph for this function:

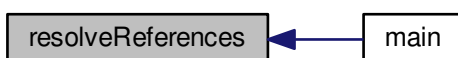


5.1.1.12 void resolveReferences ()

Here is the call graph for this function:

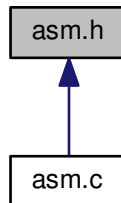


Here is the caller graph for this function:



5.2 asm.h File Reference

This graph shows which files directly or indirectly include this file:



Classes

- struct [instructionName](#)
- struct [label](#)
- struct [ref](#)

Macros

- #define [MAX_CODESEGMENT_SIZE](#) 1000
- #define [MAX_LABELS_SIZE](#) 100
- #define [MAX_IDENTS_SIZE](#) 100

Functions

- void [addCode](#) (int v)
Complète le tableau codeSegment.
- void [addInstructionName](#) (char *instname, int opcod, int type, char *format, int nbops)
- int [findLabel](#) (char *labelname)
- void [addLabel](#) (char *labelname, int addr)
- void [addReference](#) (char *labelname, int addrInCode)
- void [resolveReferences](#) ()
- void [decodeInstruction](#) (char *line)
- int [parseAsm](#) (FILE *fin)
- void [printLabels](#) ()
- void [dumpBinaryCode](#) ()
- void [generateBinary](#) (FILE *fout)

Variables

- unsigned int `codeSegment` [MAX_CODESEGMENT_SIZE]
Variable globale stockant les OPCODES.
- int `currentInst`
Variable globale indiquant le nombre courant de OPCODES dans codeSegment.
- struct `instructionName` `tabInstructionNames` [MAX_IDENTS_SIZE]
- int `currentInstructionName`
- struct `label` `tabLabels` [MAX_LABELS_SIZE]
- int `currentLabel`
- struct `ref` `tabReferences` [MAX_LABELS_SIZE]
- int `currentRef`

5.2.1 Macro Definition Documentation

5.2.1.1 `#define MAX_CODESEGMENT_SIZE 1000`

5.2.1.2 `#define MAX_IDENTS_SIZE 100`

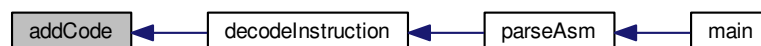
5.2.1.3 `#define MAX_LABELS_SIZE 100`

5.2.2 Function Documentation

5.2.2.1 `void addCode ( int v )`

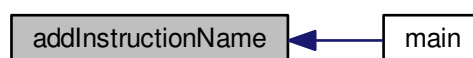
Complète le tableau `codeSegment`.

Here is the caller graph for this function:



5.2.2.2 `void addInstructionName ( char * instname, int opcod, int type, char * format, int nbops )`

Here is the caller graph for this function:

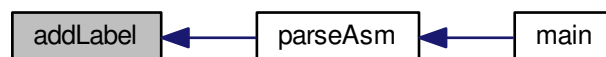


5.2.2.3 void addLabel (char * *labelname*, int *addr*)

Here is the call graph for this function:

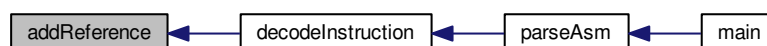


Here is the caller graph for this function:



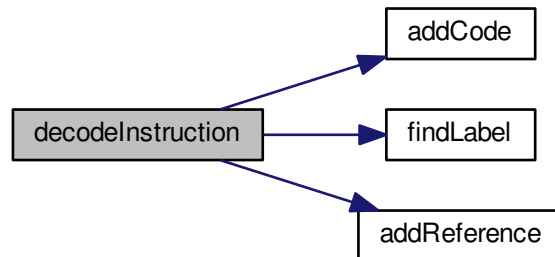
5.2.2.4 void addReference (char * *labelname*, int *addrInCode*)

Here is the caller graph for this function:

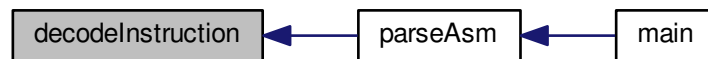


5.2.2.5 void decodeInstruction (char * line)

Here is the call graph for this function:

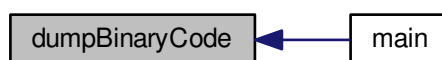


Here is the caller graph for this function:



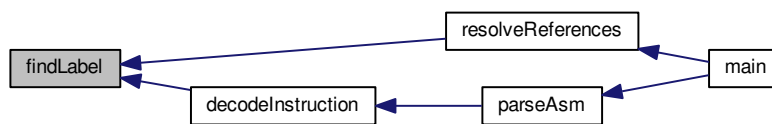
5.2.2.6 void dumpBinaryCode ()

Here is the caller graph for this function:



5.2.2.7 int findLabel (char * *labelname*)

Here is the caller graph for this function:



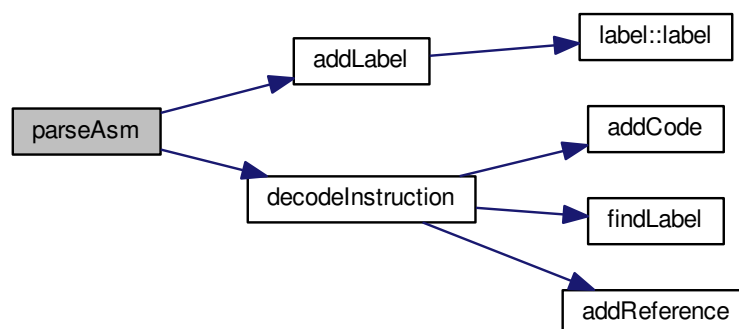
5.2.2.8 void generateBinary (FILE * *fout*)

Here is the caller graph for this function:



5.2.2.9 int parseAsm (FILE * *fin*)

Here is the call graph for this function:



Here is the caller graph for this function:



5.2.2.10 void printLabels ()

Here is the caller graph for this function:

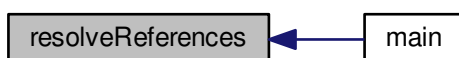


5.2.2.11 void resolveReferences ()

Here is the call graph for this function:



Here is the caller graph for this function:



5.2.3 Variable Documentation

5.2.3.1 unsigned int codeSegment[MAX_CODESEGMENT_SIZE]

Variable globale stockant les OPCODES.

5.2.3.2 int currentInst

Variable globale indiquant le nombre courant de OPCODES dans codeSegment.

5.2.3.3 int currentInstructionName

5.2.3.4 int currentLabel

5.2.3.5 int currentRef

5.2.3.6 struct instructionName tabInstructionNames[MAX_IDENTS_SIZE]

5.2.3.7 struct label tabLabels[MAX_LABELS_SIZE]

5.2.3.8 struct ref tabReferences[MAX_LABELS_SIZE]

Index

- addCode
 - asm.c, 10
 - asm.h, 16
- addInstructionName
 - asm.c, 10
 - asm.h, 16
- addLabel
 - asm.c, 10
 - asm.h, 16
- addReference
 - asm.c, 11
 - asm.h, 17
- addr
 - label, 8
- addrInCode
 - ref, 8
- asm.c, 9
 - addCode, 10
 - addInstructionName, 10
 - addLabel, 10
 - addReference, 11
 - decodeInstruction, 11
 - dumpBinaryCode, 11
 - findLabel, 12
 - generateBinary, 12
 - main, 12
 - parseAsm, 13
 - printLabels, 14
 - resolveReferences, 14
- asm.h, 15
 - addCode, 16
 - addInstructionName, 16
 - addLabel, 16
 - addReference, 17
 - codeSegment, 21
 - currentInst, 21
 - currentInstructionName, 21
 - currentLabel, 21
 - currentRef, 21
 - decodeInstruction, 17
 - dumpBinaryCode, 18
 - findLabel, 18
 - generateBinary, 19
 - MAX_CODESEGMENT_SIZE, 16
 - MAX_IDENTS_SIZE, 16
 - MAX_LABELS_SIZE, 16
 - parseAsm, 19
 - printLabels, 20
 - resolveReferences, 20
 - tabInstructionNames, 21
 - tabLabels, 21
 - tabReferences, 21
- codeSegment
 - asm.h, 21
- currentInst
 - asm.h, 21
- currentInstructionName
 - asm.h, 21
- currentLabel
 - asm.h, 21
- currentRef
 - asm.h, 21
- decodeInstruction
 - asm.c, 11
 - asm.h, 17
- dumpBinaryCode
 - asm.c, 11
 - asm.h, 18
- findLabel
 - asm.c, 12
 - asm.h, 18
- format
 - instructionName, 7
- generateBinary
 - asm.c, 12
 - asm.h, 19
- instructionName, 7
 - format, 7
 - name, 7
 - nbops, 7
 - opcod, 7
 - type, 7
- label, 7
 - addr, 8
 - label, 8
 - ref, 8
- MAX_CODESEGMENT_SIZE
 - asm.h, 16
- MAX_IDENTS_SIZE
 - asm.h, 16
- MAX_LABELS_SIZE
 - asm.h, 16
- main

- asm.c, [12](#)
- name
 - instructionName, [7](#)
- nbops
 - instructionName, [7](#)
- opcod
 - instructionName, [7](#)
- parseAsm
 - asm.c, [13](#)
 - asm.h, [19](#)
- printLabels
 - asm.c, [14](#)
 - asm.h, [20](#)
- ref, [8](#)
 - addrInCode, [8](#)
 - label, [8](#)
- resolveReferences
 - asm.c, [14](#)
 - asm.h, [20](#)
- tabInstructionNames
 - asm.h, [21](#)
- tabLabels
 - asm.h, [21](#)
- tabReferences
 - asm.h, [21](#)
- type
 - instructionName, [7](#)